

Dashboard guide

What each of the eleven dashboards built for the account shows, what it is for, where its data comes from and what it does not cover yet.

DATE	ACCOUNT	MANAGED BY
25 September 2026	1468011 (US region)	Terraform — repo <code>ipayout-newrelic</code>

How this is organised

The account holds twenty-one dashboards. Ten already existed and were imported as they were; the remaining eleven are the ones we built, and those are what this document covers.

All of them are defined in code, with Terraform, in the `ipayout-newrelic` repository. Each dashboard lives in its own `.tf` file, which is the single source of truth: to change a widget you edit that file, run `terraform plan` to see the difference and `terraform apply` to apply it. Editing a dashboard from the New Relic UI works until the next `apply` reverts it; in exchange, every dashboard is reproducible and carries the record of why it looks the way it does.

They fall into three families: six look at the whole account and in practice mostly show production, four are scoped to the Test environment — which an account-wide dashboard buries under production traffic — and one is the production twin of the critical-services board.

INDEX

DASHBOARD	ANSWERS	DATA SOURCE
WHOLE ACCOUNT		
Infrastructure - Hosts (CPU, Memory, Disk)	Is any server running out of CPU, memory or disk?	Infrastructure agent
Host Processes	Which process is eating that host?	Infrastructure agent
Web Availability (Synthetics)	Are the sites up, and how fast do they answer?	135 ping monitors
SQL Server	How are the database instances doing?	nri-mssql integration
APM - Application Health	How are the .NET applications performing?	APM agents
Errors & Exceptions	What is failing, with which exception, in which app?	APM agents
TEST ENVIRONMENT		
Test Environment - Health Overview	Is Test healthy right now?	All three sources
Windows Services - Test Environment	Which Windows services run, and which stopped?	nri-winservices integration
APM - Test Environment	How are the Test applications performing?	APM agents
Test - Critical Services Uptime	Did the services required at 100% stay at 100%?	nri-winservices integration
PRODUCTION		
Production - Critical Services Uptime	The same, for the production hosts	nri-winservices integration

Whole account

1 Infrastructure - Hosts (CPU, Memory, Disk)

dashboard_hosts.tf

What it is for: the front page of infrastructure. It answers whether any server is about to run out of CPU, memory or disk, and whether that is getting worse or is just a spike.

What it shows	Three tables with the current state — CPU % per host, memory % per host, and disk % per host and volume — and three trend charts with the same metrics over time.
Source	Infrastructure agent: <code>SystemSample</code> for CPU and memory, <code>StorageSample</code> for disk.
Scope	The whole account, with no fixed host filter. All 27 reporting servers appear on their own, and a new host shows up without touching the code.
How to read it	The <code>Host</code> selector at the top is multi-select and filters the whole board. The tables answer "how is it now?"; the lines answer "has it been climbing?".

The detail that matters: disk is broken out per host *and per volume*, so a server with C:, D: and E: appears three times. That is deliberate — disk problems are almost always one specific volume, like the P: drive on UE2-SQL-A01 sitting at 96%.

2 Host Processes

dashboard_processes.tf

What it is for: the next step after the previous board. When that one flags a host on high CPU or memory, this one says which process is causing it.

What it shows	Top 25 processes by CPU and top 25 by memory, in tables broken out by host and process name, plus trends for the top 10 of each.
Source	Infrastructure agent: <code>ProcessSample</code> .
Scope	The whole account, with the same <code>Host</code> selector.
How to read it	Pick the suspect host in the selector first; unfiltered, the tables mix processes from different servers and stop meaning anything.

Cost: `ProcessSample` is one of the most expensive events in the account's ingest, because it sends one sample per process per host. If it needs trimming, `scripts/Set-ProcessSampleRate.ps1` lowers the sampling rate without losing the dashboard.

3 Web Availability (Synthetics)

dashboard_synthetics.tf

What it is for: the outside view. It says whether the sites respond and how long they take, without depending on the server being instrumented from the inside.

What it shows Four indicators at the top — sites monitored, sites failing in the last 30 minutes, 24-hour uptime % and average response time — and below them a table of uptime and latency per site, another of failures with the last error received, the trend for the 20 slowest sites, and the curve of failed checks.

Source SyntheticCheck , fed by the 135 monitors defined in synthetics.tf .

The monitors Simple ping every 10 minutes from two regions (US East and US West), with SSL verification, against the /public/healthcheck.ashx endpoint. The exception is supernova.i-payout.com , which has no such endpoint and is checked against its homepage.

How to read it The failing-sites indicator turns red at one. The failures table carries the error message, which usually distinguishes an expired certificate from a real outage.

Limitation: the 135 monitors have no alert conditions yet. This board has to be looked at; for now it does not notify on its own.

4 SQL Server

dashboard_mssql.tf

What it is for: the health of the database instances, with enough detail to tell "the database is slow" apart from "the database is blocked".

What it shows Four indicators — active connections, blocked processes, deadlocks per second and buffer pool hit rate; trends for memory, page life expectancy, transactions per second and lock waits; the top 15 wait types with how they evolve; and two per-database tables with I/O stalls and buffer pool consumed.

Source nri-mssql integration: MssqlInstanceSample , MssqlDatabaseSample and MssqlWaitSample .

How to read it Blocked processes and deadlocks turn red at one: any non-zero value is worth a look. Page life expectancy falling is the classic early sign of memory pressure, before it shows up as latency.

Incomplete coverage: only UE2-SQL-A01 and MIAT-VM-SQL-001 report today. Missing are UE2-SQL-A02 , UE2-SQL-B02 and BCA-VM-SQL-001 , which need nri-mssql installed. Separately, UE1-TEST-SQL-A2 and UE1-TEST-SQL-B2 run SQL Server 2025, which the integration does not support yet: those two cannot appear until New Relic adds support.

5 APM - Application Health

dashboard_apm.tf

What it is for: the golden signals of the roughly twenty instrumented .NET applications — how much traffic they handle, how long they take and how much they fail — on one screen.

What it shows Four global indicators (transactions per minute, average time, 95th percentile and error rate), the same three signals broken out per application over time, a comparison table of every app, and two transaction tables: the slowest and the most called.

Source APM agents: the `Transaction` event.

Thresholds Error rate turns amber at 1% and red at 5%.

How to read it Average and p95 together say more than either alone: if the average looks fine and p95 spikes, the problem hits a minority of requests, not all of them.

Scope: it looks at the whole account, so production dominates — IPS New alone does on the order of a million transactions a day. To see Test, use the twin dashboard (number 9), where that volume hides nothing.

6 Errors & Exceptions

dashboard_errors.tf

What it is for: the complement of the previous one. APM says *how much* is failing; this says *what* is failing: which exception, with which message, in which application.

What it shows Total errors, how many applications are affected and the overall error rate; errors per app over time; a per-application table with the last message received; a ranking by exception class; and a table of the 25 most frequent messages, broken out by app, class and text.

Source APM agents: `TransactionError` for the detail and `Transaction` to compute the percentage.

How to read it Start from the exception-class ranking: it usually concentrates most errors into two or three types, and the message table below gives the exact text to search for in the code.

Open finding: when first checked (31 August 2026), `CommissionNetworks.com` had an error rate close to 58%.

Test environment

The four Test boards exist because in an account-wide dashboard the test environment disappears underneath production volume. All of them carry their scope fixed in code — `hostname LIKE 'UE1-TEST%'` and equivalents — injected into every query, so they do not depend on anyone getting the filter right by hand.

7 Test Environment - Health Overview

`dashboard_test_overview.tf`

What it is for: the one screen to look at to know whether Test is fine. The other three boards each answer one question well; this one answers "is the environment healthy right now?", which needs all three sources at once.

How it is built The top row is the only part you read when nothing is wrong: hosts reporting out of the 11 in the environment, hosts above 85% memory, hosts above 85% disk, IPS services stopped, application error rate and throughput. Everything below is the detail behind one of those six numbers.

Sections *Host resources:* a per-host health table with CPU, memory, uptime in days and agent version, plus disk per volume with free GB alongside, and trends. *IPS services:* which ones are stopped and how many per host. *Applications:* a per-application table with latency and errors. *Coverage:* which instrumentation layer reaches each host.

Source All three at once: `SystemSample` and `StorageSample` from the agent, `windows_service_state` from nri-winservices, and `Transaction` from APM.

TWO DESIGN DECISIONS WORTH KNOWING

It measures its own blind spot. A dashboard showing only the hosts that report would look complete while omitting precisely the ones sending nothing. That is why the tile reads "Hosts reporting (of 11)": it compares against the known size of the environment, not against whatever happened to arrive.

It carries no hand-written host lists. The first version of the coverage section named the five hosts not reporting on 15 September; four came online the next day and the text sat there wrong for days before anyone noticed. Now the numbers always come from the data and the text only explains how to read them.

About the thresholds: 85% for memory and disk is not a round number picked at random — it comes from what the fleet actually runs at (between 50% and 91% memory), so it flags the genuinely tight cases without lighting up on normal Windows behaviour. Disk has to be read alongside the `Free GB` column: 87% of a 267 GB volume leaves 34 GB and is fine, while 85% of a 32 GB system drive leaves under 5 GB and is not.

8 Windows Services - Test Environment

dashboard_windows_test.tf

What it is for: the state of every Windows service on the Test hosts — which are running, which are stopped and which changed state recently. It mirrors the production "Windows Services - Dashboard", but with its scope pinned.

What it shows A summary (hosts, IPS services not running, services by state); a critical-services section with how many are stopped and which ones; services running, stopped and paused per host, each with its bar and its trend; the full service inventory; state transitions over the last 15 minutes in two tables, *Stopped* and *Started*; and breakdowns by service account, by start mode and by states other than the three usual ones.

Source `nri-winservices` integration: the `windows_service_state` metric.

Scope `hostname LIKE 'UE1-TEST%' OR hostname LIKE 'MIAT-VM%'`, injected into all sixteen queries. The production board, by contrast, has no scope and shows whatever reported.

Filters Five combinable variables: `Host`, `Service name`, `Display name`, `State` and `Start Mode`.

WHY THE TIME WINDOWS ARE WHAT THEY ARE

The Test hosts are scraped every 400 seconds, a cost decision: collecting *every* service at that interval costs on the order of thirteen times less than doing it every 30 seconds, and it avoids having to curate a service list. The board's windows are tuned to that rhythm. The variables use one day, because at three days or more New Relic reads pre-aggregated rollups that lag several hours behind: a freshly onboarded host would return nothing, the variable would fall back to its default, and every widget would render zero. The transition tables use fifteen minutes, because five is shorter than the sampling interval and they would almost always be empty.

Caveat: the "IPS services not running" tile uses a hand-written name list that has drifted from the real service names, so it *undercounts*. The critical-services section, by contrast, matches by pattern (`service_name LIKE 'ips%'`) and does not have that problem. For anything critical, use that section or dashboard number 10.

9 APM - Test Environment

dashboard_apm_test.tf

What it is for: the same golden signals as the APM dashboard, but scoped to the `UE1-TEST-*` hosts, which are invisible in the account-wide board next to production traffic.

What it shows Six indicators at the top (throughput, average time, 95th percentile, error rate, application count and hosts reporting); the signals broken out per host and per application over time; a comparison table of applications; rankings of the slowest and of the highest error rate; and the transaction and error detail.

Source APM agents: `Transaction` and `TransactionError`.

Filters

Application and Host variables.

A tile that doubles as a rollout tracker: "Hosts reporting APM (of 4)" reads **2**, and will keep reading 2 until the .NET agent is installed on the two web hosts. `UE1-TEST-WEB-A1` runs IIS but has no .NET agent, and `UE1-TEST-WEB-B1` has no agent at all. The number rises on its own as the rollout progresses.

10 Test - Critical Services Uptime

`critical_services.tf`

What it is for: tracking the services the business requires at 100% availability. It is the only one of the eleven dashboards with an alert behind it: the others have to be watched, this one notifies.

What it shows	Critical services not running, hosts that run them, services reporting and combined uptime; a table with the uptime % and current state of each host + service pair; and the trend of that uptime over time.
Which services	<code>ipsportalcoreschedulerservice</code> , <code>ips_abacollectorservice</code> , <code>ips_emailsenderservice</code> , <code>ips_feecollector</code> , <code>ips_fxservice</code> , <code>ips_healthmonitor</code> , <code>ips_loadvirtualaccountsservice</code> and both names Ripple reports under (<code>ripplepaymentsservice</code> on Test, <code>ipsripplepaymentsservice</code> in production; both are listed so a renamed deployment stays covered).
Scope	Every UE1-TEST host. A facet only forms where a service actually reports, so hosts that run none of them never create alert targets. Today only <code>UE1-TEST-SRV-A1</code> runs them, with all eight active; any Test host that picks one up is covered automatically.
What uptime means	The percentage of samples in the <code>running</code> state within the selected time range, at the resolution of the 400-second scrape. It is not continuous measurement: it is the sampling every 400 s.

THE ALERT BEHIND IT

The policy is called `Test - Critical services` and its condition "Critical service is not running on a Test host". It opens one incident per host and service combination, aggregates in 600-second windows and notifies Slack in `#newrelic-errors` . Loss of signal also triggers an alert: if a host stops reporting, that is announced rather than passing unnoticed. In practice a stopped service reaches Slack within about 10 to 20 minutes — the price of having chosen a 400-second scrape over a 30-second one.

Production

11

Production - Critical Services Uptime

critical_services.tf

What it is for: the same critical-services board, pointed at the production hosts `BCA-VM-*` and `UE2-*`.

What it shows	Exactly the same six widgets as its Test twin, with an identical service list and the scope changed.
Key difference	It has no alert wired to it. It is visibility only: the critical-services alert covers Test exclusively. Extending it to production was proposed and deferred (23 September 2026).
Current coverage	The only production host with data is <code>BCA-VM-SRV-001</code> , which runs seven of the listed services — it does not run <code>ips_healthmonitor</code> . The rest of production will appear as the <code>nri-winservices</code> rollout progresses.

Appendices

APPENDIX A

Where the data comes from

The eleven dashboards feed on four sources. Each one needs something different on the servers, and that is where the coverage gaps come from.

SystemSample · **StorageSample** · **ProcessSample**

Infrastructure agent. The base layer: CPU, memory, disk and processes. Present on all 27 hosts in the account.

Transaction · **TransactionError**

APM agents (.NET). They require installing the agent inside the application. They cover the ~20 production apps and 2 of the 4 Test hosts.

windows_service_state (Metric)

nri-winservices integration. Bundled with the Windows agent — nothing to install: it is enabled by dropping a config file and restarting the agent, which is what `scripts/Enable-NriWinservices.ps1` automates.

MssqlInstanceSample · **MssqlDatabaseSample** · **MssqlWaitSample**

nri-mssql integration. Unlike the previous one, this does have to be installed separately on each database host.

SyntheticCheck

Synthetic monitors. They need nothing on the servers: they run from New Relic's own infrastructure and look at the sites from outside.

APPENDIX B

What is missing for the boards to fill up

State as of 25 September 2026. None of these gaps is a dashboard problem: they are servers the instrumentation has not reached yet.

LAYER	COVERAGE	WHAT IS MISSING
Infrastructure agent	27 of 27 hosts	Nothing. The only complete layer.
nri-winservices	11 of 27 hosts	Both MIAT-VM hosts and 19 of the 20 in production. Affects dashboards 8, 10 and 11.
nri-mssql	2 instances	UE2-SQL-A02, UE2-SQL-B02 and BCA-VM-SQL-001. The two Test SQL hosts run SQL Server 2025 and are blocked until the integration supports it.
.NET agent (APM)	2 of 4 Test hosts	UE1-TEST-WEB-A1 (runs IIS with no agent) and UE1-TEST-WEB-B1 (no agent).
Alerts on synthetics	0 of 135 monitors	Dashboard 3 shows the outages but does not notify yet.